Safe Robotic Policy Learning via Manifold Safety Filter and Offline Reachability Analysis

Anonymous IROS 2026 Submission

***Abstract*— Reinforcement learning enables adaptive robotic control, yet deploying learned policies in safety-critical environments demands provable guarantees. Existing safe RL methods either employ soft constraints whose violation rates are sensitive to penalty hyperparameters, or rely on online environmental sensing that is difficult to guarantee in practice. We propose a unified framework that addresses both limitations. First, we develop a manifold safety filter that projects policy actions onto the tangent space of the constraint manifold via null-space projection, providing provable hard safety guarantees. A dynamic reward calibration mechanism further guides the policy to learn inherently safe behaviors by penalizing the correction magnitude. Second, we introduce offline Hamilton-Jacobi reachability analysis to pre-compute environment-aware safe regions from collected trajectory data, eliminating the need for real-time obstacle sensing at deployment. Third, we design a data-driven Extended Kalman Filter with CNN-learned noise parameters that adapts measurement covariance to motion dynamics, ensuring robust state estimation under velocity-dependent sensor noise. We provide theoretical proof that the manifold projection guarantees constraint satisfaction under mild regularity conditions. Experiments on Safety-Gymnasium demonstrate that our method maintains consistently low constraint violations across varying penalty settings, while baselines fluctuate by up to an order of magnitude. Ablation studies confirm each component's contribution, and zero-shot deployment on a simulated E-puck robot in Webots achieves zero collisions across 20 trials, demonstrating cross-platform transferability.**

I. INTRODUCTION

Safety constraints are fundamental requirements for deploying robots in real-world environments such as autonomous driving and service robotics. However, ensuring robot safety remains challenging due to environmental complexity, dynamic obstacles, and sensor noise. Reinforcement Learning (RL) has gained significant attention for its ability to learn adaptive control policies through environmental interaction [1]. However, the trial-and-error nature of RL training poses inherent safety risks during both training and deployment.

Safe Reinforcement Learning (SafeRL) addresses this challenge by formalizing RL tasks as Constrained Markov Decision Processes (CMDPs) [3]. Existing approaches can be categorized into two main classes:

Cost-based methods balance reward and cost through adaptive penalties. PID Lagrangian [4], interior-point [5], and soft barrier [6] methods improve constraint satisfaction but employ *soft constraints* that often fail to achieve zero-cost performance [1].

State-constrained methods enforce safety via CBFs [7], [18], recovery policies [19], or constraint manifold projection [8], [9]. However, they typically require *online* environmental sensing and accurate state estimation, both difficult to guarantee in practice.

Contributions. This paper proposes a unified framework that eliminates the need for online environmental sensing in state-constrained safe RL, with three main contributions:

1) **Constraint Manifold Filter with Reward Calibration**: We develop a null-space projection method that transforms any policy action into a provably safe action by projecting onto the tangent space of the constraint manifold (Section IV-A). Unlike prior work [8], our method incorporates dynamic reward calibration that penalizes correction magnitude, guiding the policy to learn inherently safe behaviors.
2) **Offline Safe Region Construction**: We introduce *offline* Hamilton-Jacobi reachability analysis to pre-compute environment-aware safe regions from collected trajectory data, enabling the safety filter to operate without real-time obstacle detection (Section IV-B). This decouples safety from online perception—the key enabler for sensing-free deployment.
3) **Data-Driven State Estimation**: We propose a data-driven EKF with CNN-learned noise parameters that adapts measurement covariance to motion dynamics, maintaining safety filter accuracy under velocity-dependent sensor noise (Section IV-C).

We provide theoretical guarantees for constraint satisfaction and validate our framework through extensive experiments on Safety-Gymnasium [16] and Webots simulation, demonstrating significant cost reduction across navigation tasks while maintaining competitive reward performance.

II. RELATED WORK

Safe RL. Recent surveys [1], [2] categorize SafeRL into cost-based and state-constrained approaches. Cost-based methods optimize a penalty-augmented objective: PID Lagrangian [4] adapts multipliers via a PID controller, IPO [5] uses log-barrier penalties, and soft barrier methods [6] maintain feasibility probabilistically. While these approaches are general-purpose, they provide *soft* constraint satisfaction—the violation rate depends critically on penalty hyperparameters, and zero-cost performance is rarely achieved [1].

State-constrained methods enforce safety through action-space restrictions. Control Barrier Functions (CBFs) [7], [18] modify actions to keep the system within a forward-invariant safe set, but require a valid CBF—often difficult to construct for complex environments. Recovery RL [19] learns recovery zones offline and switches to a learned recovery policy when

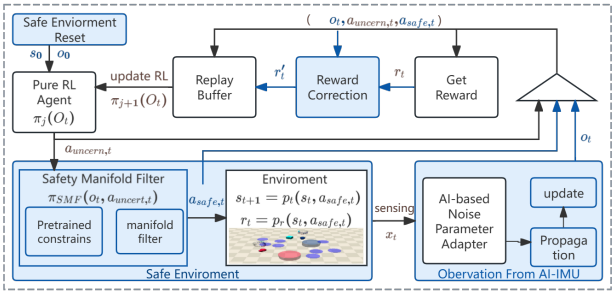


Fig. 1: Framework overview. The RL policy π_θ outputs an unconstrained action $\alpha \in \mathbb{R}^m$, which is projected onto the constraint manifold tangent space by the safety filter to produce a safe action a_{safe}. The data-driven EKF processes noisy sensor observations to provide accurate state estimates for both the policy and safety filter.

the agent approaches unsafe regions, but its safety depends on the quality of the learned recovery boundary.

ATACOM [8], [9] takes a different approach by projecting actions onto the tangent space of the constraint manifold via null-space projection, providing provable constraint satisfaction for robotic manipulation tasks. However, ATACOM assumes *accurate* state information and *known* obstacle positions at each timestep—requirements that are difficult to guarantee in practice with noisy sensors and partially observable environments. Our work extends this manifold projection paradigm in three ways: (i) offline reachability analysis eliminates the need for real-time obstacle sensing, (ii) data-driven EKF provides robust state estimation under sensor noise, and (iii) reward calibration guides the policy to learn inherently safe behaviors.

HJ Reachability. Hamilton-Jacobi reachability analysis [10] computes backward reachable sets by solving the HJ partial differential equation, providing formal safety guarantees. Recent work addresses scalability: neural network approximations [11], [12] enable reachability computation in higher dimensions. However, most methods require online computation or access to the environment model. We instead adopt an offline approach inspired by IQL [15], pre-computing feasibility value functions from collected trajectory data. This decouples safety verification from deployment, enabling the safety filter to operate without real-time obstacle sensing—a key requirement for resource-constrained robotic platforms.

Learned State Estimation. Classical EKF assumes known, fixed noise statistics, which may not hold under varying operating conditions. Learning-based methods address this limitation: deep IMU bias inference [13] learns IMU biases for visual-inertial odometry, while TLIO [14] uses CNNs for tight inertial odometry. KalmanNet [21] learns Kalman gains end-to-end, and A-KIT [22] uses transformers for adaptive process noise estimation. Our CNN-based noise adapter is lightweight (5K parameters) and specifically designed to learn velocity-dependent measurement covariance from IMU windows, providing an adaptive R_t that improves state estimation accuracy for the downstream safety filter.

III. PROBLEM FORMULATION

A. Safe Robot Navigation

Consider a robot navigating in an environment with n obstacles. Let $s_t = [p_t^\top, v_t^\top]^\top \in \mathcal{S} \subseteq \mathbb{R}^d$ denote the robot state at discrete time t, where $p_t \in \mathbb{R}^{d_p}$ is the position and $v_t \in \mathbb{R}^{d_p}$ is the velocity. We define safety through distance-based constraints:

$$c_i(s_t) := d_{\text{safe}} - \|p_t - o_i\|_2 \leq 0, \quad \forall i \in \{1, \ldots, n\} \tag{1}$$

where $d_{\text{safe}} > 0$ is the minimum safety distance, $o_i \in \mathbb{R}^{d_p}$ denotes the position of obstacle i, and $\|\cdot\|_2$ denotes the Euclidean norm. A state s_t is *safe* if $c_i(s_t) \leq 0$ for all i, meaning the robot maintains at least distance d_{safe} from all obstacles.

B. Constrained Markov Decision Process

We formalize safe RL as a CMDP $(\mathcal{S}, \mathcal{A}, P, R, \gamma, C)$ with state space $\mathcal{S}$, action space $\mathcal{A}$, transition kernel P, reward R, discount γ, and constraint set $C = \{c_i : \mathcal{S} \to \mathbb{R}\}_{i=1}^k$. The objective is to find $\pi : \mathcal{S} \to \mathcal{A}$ maximizing expected return under *hard constraints*:

$$\max_\pi \mathbb{E}_\pi \left[\sum_{t=0}^{T} \gamma^t R(s_t, a_t) \right] \quad \text{s.t.} \quad c_i(s_t) \leq 0, \ \forall t, i \tag{2}$$

C. Control-Affine System Dynamics

Due to the prevalence of nonholonomic constraints in robotic systems, we model the dynamics as a control-affine system:

$$\dot{s} = f(s) + G(s)a \tag{3}$$

where $a \in \mathcal{A} \subseteq \mathbb{R}^m$ is the m-dimensional control action, $f : \mathcal{S} \to \mathbb{R}^d$ is the drift term representing uncontrolled dynamics, and $G : \mathcal{S} \to \mathbb{R}^{d \times m}$ is the control input matrix mapping actions to state derivatives. For a point robot with single-integrator dynamics, $f(s) = 0$ and $G(s) = I$; for a differential-drive robot, $G(s)$ encodes the kinematic constraints.

IV. METHOD

A. Constraint Manifold Filter

A manifold is a topological space locally homeomorphic to Euclidean space. We define the *constraint manifold* as the set of configurations satisfying all safety constraints.

1) Slack Variable Formulation: To handle inequality constraints, we introduce slack variables $\mu \in \mathbb{R}^k_{>0}$ that transform inequalities into equalities. The augmented constraint function becomes:

$$\bar{c}(s, \mu) := c(s) + \phi(\mu) = 0 \tag{4}$$

where $\phi : \mathbb{R}^k_{>0} \to \mathbb{R}^k_{<0}$ is a strictly negative penalty function. We use the *softcorner* formulation:

$$\phi(\mu) = -\frac{1}{\beta} \ln(-\text{expm1}(\beta\mu)) \tag{5}$$

where $\beta > 0$ controls the sharpness of the penalty. The constraint manifold is then:

$$\mathcal{M} := \{(s, \mu) \in \mathcal{D} \mid \bar{c}(s, \mu) = 0\} \tag{6}$$

where $\mathcal{D} = \mathcal{S} \times \mathbb{R}^k_{>0}$ is the augmented state space.

2) Null-Space Projection: Let $\alpha \in \mathbb{R}^{m+k}$ denote the augmented action vector output by the RL policy, where m is the control dimension and k is the number of constraints (slack variables). The safety filter transforms α into a safe action $[a_{\text{safe}}^\top, \dot{\mu}^\top]^\top$ that satisfies all constraints.

Taking the time derivative of the constraint:

$$\frac{d}{dt}\bar{c}(s, \mu) = J_s \dot{s} + J_\mu \dot{\mu} = J_s f(s) + J_c \begin{bmatrix} a \\ \dot{\mu} \end{bmatrix} \tag{7}$$

where $J_s = \frac{\partial \bar{c}}{\partial s}$, $J_\mu = \frac{\partial \bar{c}}{\partial \mu}$, and $J_c = [J_s G(s), J_\mu] \in \mathbb{R}^{k \times (m+k)}$ is the combined constraint Jacobian.

To maintain the system on the constraint manifold, we require $\frac{d}{dt}\bar{c} = 0$. Using the damped pseudoinverse $J_c^+ = J_c^\top (J_c J_c^\top + \epsilon I)^{-1}$ with regularization $\epsilon > 0$, the null-space projector is:

$$N_c = I - J_c^+ J_c \tag{8}$$

Given an unconstrained action α from the RL policy, the safe action is computed as:

$$\begin{bmatrix} a_{\text{safe}} \\ \dot{\mu} \end{bmatrix} = N_c \alpha - K_c J_c^+ \bar{c}(s, \mu) - J_c^+ J_s f(s) \tag{9}$$

where $K_c > 0$ is a correction gain. The first term projects the policy action onto the constraint null-space (tangent space of $\mathcal{M}$), the second drives the state toward the manifold when deviations occur, and the third compensates for drift dynamics $f(s)$.

Theorem 1 (Safety Guarantee): If the initial state satisfies $c(s_0) \leq 0$ and the action is computed via (9), then $c(s_t) \leq 0$ for all $t \geq 0$.

Proof: Substituting (9) into (7): the null-space term $N_c \alpha$ satisfies $J_c N_c = 0$ by construction, the drift term cancels $J_s f(s)$, and the correction term yields $\dot{\bar{c}} = -K_c \bar{c}$. This gives exponential convergence $\bar{c}(t) = \bar{c}(0) e^{-K_c t}$. Since $\phi(\mu) < 0$ for all $\mu > 0$, we have $c(s) = \bar{c} - \phi(\mu) < \bar{c}(0) e^{-K_c t} \leq 0$ when initialized feasibly. ■

Remark (Discrete-time implementation). In practice, we apply (9) at each discrete timestep Δt. The discretization introduces bounded error $O(\Delta t^2)$ per step; the correction term $-K_c J_c^+ \bar{c}$ actively compensates for accumulated drift, ensuring constraint satisfaction is maintained for sufficiently small Δt relative to K_c.

3) Reward Calibration: We calibrate the reward based on correction magnitude:

$$R_{\text{calibrated}} = R(s_t, a_{\text{safe}}) - \lambda \| a_{\text{unsafe}} - a_{\text{safe}} \|_2^2 \tag{10}$$

where $\lambda > 0$ is the penalty weight, a_{unsafe} is the original policy output, and a_{safe} is the projected action. This encourages the policy to learn inherently safe behaviors, minimizing the need for correction.

Algorithm 1 Safe RL with Constraint Manifold Projection

Input: Constraint function $c(s)$, gains K_c, λ
Initialize: Policy π_θ, feasible state s_0, slack μ_0
for each episode **do**
  for each timestep t **do**
    Sample policy action: $\alpha_t \sim \pi_\theta(\cdot|s_t)$
    Compute Jacobians: J_s, J_μ, J_c, N_c
    Project to safe action via (9): a_{safe}
    Execute a_{safe}, observe s_{t+1}, r_t
    Compute calibrated reward via (10)
    Store transition, update policy
  end for
end for

B. Safe Reachability Analysis

The constraint manifold filter requires knowledge of obstacle positions. To construct environment-aware safe regions offline, we employ Hamilton-Jacobi (HJ) reachability analysis [10].

1) Feasibility Value Functions: We define the optimal feasible state-value function as the minimum over all policies of the maximum constraint violation:

$$V_c^*(s) := \min_\pi \max_{t \in \mathbb{N}} c(s_t), \quad s_0 = s \tag{11}$$

A state s is *feasible* if $V_c^*(s) \leq 0$, meaning there exists at least one policy that satisfies hard constraints from s. The maximal feasible region is:

$$\mathcal{S}_f^* := \{s \in \mathcal{S} \mid V_c^*(s) \leq 0\} \tag{12}$$

2) Offline Approximation: We approximate V_c^* using the feasible Bellman operator:

$$\mathcal{T}^* Q_c(s, a) := (1 - \gamma) c(s) + \gamma \max\{c(s), V_c(s')\} \tag{13}$$

We use expectile regression [15] to handle out-of-distribution actions:

$$L_{V_c} = \mathbb{E}_{(s,a) \sim \mathcal{D}} \left[L_{\text{rev}}^\tau (Q_c(s, a) - V_c(s)) \right] \tag{14}$$

$$L_{Q_c} = \mathbb{E}_{(s,a,s') \sim \mathcal{D}} \left[(\mathcal{T}^* Q_c - Q_c(s, a))^2 \right] \tag{15}$$

where $L_{\text{rev}}^\tau(u) = |\tau - \mathbf{1}(u > 0)| \cdot u^2$ with $\tau \in [0.5, 1]$ gives higher weight to smaller Q_c values.

3) Integration with Safety Filter: The learned V_c^* modulates the safety filter's conservatism at deployment. Specifically, we scale the danger radius as $d_{\text{danger}}(s) = d_0 + \eta \cdot \max(V_c^*(s), 0)$, where d_0 is the base radius and $\eta > 0$ controls the margin. States near the boundary of $\mathcal{S}_f^*$ (where $V_c^* \approx 0$) receive enlarged safety margins, while states deep within the feasible region use the nominal margin. This enables the filter to be *environment-aware* without requiring real-time obstacle sensing—the pre-computed V_c^* encodes the reachability structure of the environment.

Fig. 2 visualizes the learned feasibility value function for a representative environment configuration. The value landscape reveals the safe region structure: blue regions ($V_c^* \ll 0$) indicate states far from any constraint boundary where

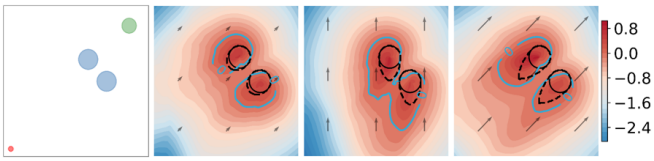


Fig. 2: Learned feasibility value function $V_c^*(s)$. Left: environment layout (blue = obstacles, green = goal, red = robot). Right three panels: value landscape from different initial configurations. Blue = safe ($V_c^* < 0$), red = infeasible ($V_c^* > 0$), cyan contour = feasible boundary $\partial\mathcal{S}_f^*$, dashed = danger zone with enlarged margins. Arrows indicate the gradient direction of increasing safety.

minimal safety intervention is needed, while red regions ($V_c^* > 0$) mark infeasible states inside or near obstacles. The cyan contour ($V_c^* = 0$) delineates the boundary of the maximal feasible region $\mathcal{S}_f^*$. The gradient field (arrows) shows the direction of increasing safety, which the filter uses to modulate the danger radius.

C. Data-Driven State Estimation

Accurate state estimation is critical for safe navigation. Position sensors (e.g., GPS, visual odometry) suffer from noise that varies with operating conditions—faster motion increases vibration, multipath effects, and motion blur, leading to *velocity-dependent* measurement uncertainty. We model this as:

$$\sigma(v) = \sigma_{\text{base}}(1 + k\|v\|) \tag{16}$$

where σ_{base} is the baseline noise standard deviation, $k > 0$ is the velocity scaling coefficient, and $\|v\|$ is the robot speed. A standard EKF with fixed noise covariance $R = \sigma_{\text{base}}^2 I$ cannot adapt to this variation, leading to over-trust of noisy measurements during fast motion. We propose a data-driven EKF that learns to predict the noise covariance from recent IMU data.

1) Extended Kalman Filter: We employ an EKF that linearizes the nonlinear system dynamics around the current state estimate:

$$\hat{s}_{t|t-1} = f(\hat{s}_{t-1}, a_{t-1}) \tag{17}$$

$$P_{t|t-1} = F_t P_{t-1} F_t^\top + Q_t \tag{18}$$

where $F_t = \frac{\partial f}{\partial s}\big|_{\hat{s}_{t-1}}$ is the Jacobian of the dynamics and Q_t is the process noise covariance.

2) Learned Noise Parameters: Rather than manually tuning the measurement noise covariance R_t, we learn it using a CNN (NoiseAdapter) that processes a sliding window of IMU measurements:

$$z_t = \text{CNN}\left(\{\omega_i^{\text{IMU}}, a_i^{\text{IMU}}\}_{i=t-L}^{t}\right) \tag{19}$$

where L is the window length and $z_t = [z_t^{\text{lat}}, z_t^{\text{up}}]^\top \in \mathbb{R}^2$. The adaptive measurement covariance is:

$$R_t = \text{diag}\left(\sigma_{\text{lat}}^2 \cdot 10^{\beta \tanh(z_t^{\text{lat}})}, \sigma_{\text{up}}^2 \cdot 10^{\beta \tanh(z_t^{\text{up}})}\right) \tag{20}$$

where $\sigma_{\text{lat}}, \sigma_{\text{up}}$ are baseline noise estimates and $\beta > 0$ controls the adaptation range. The $\tanh$ nonlinearity bounds the output, while the exponential parameterization ensures $R_t \succ 0$. When $z_t \approx 0$ (e.g., before convergence), R_t defaults to the baseline covariance, ensuring stability.

3) NoiseAdapter Pretraining: The CNN is pretrained offline on tuples $(\mathbf{w}_t, \tilde{y}_t, y_t)$ from random exploration, where $\mathbf{w}_t \in \mathbb{R}^{L\times 6}$ is the IMU window, $\tilde{y}_t$ is the noisy measurement, and y_t is ground truth. Training minimizes negative log-likelihood:

$$\mathcal{L}_{\text{NLL}} = \frac{1}{|\mathcal{D}_{\text{IMU}}|} \sum_{(\mathbf{w},\tilde{y},y)} \left[\log|R(\mathbf{w})| + (\tilde{y}-y)^\top R(\mathbf{w})^{-1}(\tilde{y}-y)\right] \tag{21}$$

This loss encourages predicting large R during high-speed motion and small R at low speed, directly learning the velocity–noise correlation.

(a) Goal (b) Circle (c) Push

Fig. 3: Evaluation tasks in Safety-Gymnasium. Red zones indicate hazards; robots must complete objectives while avoiding collisions.

V. EXPERIMENTS

We evaluate our framework on the Safety-Gymnasium benchmark [16] across three tasks with varying difficulty levels.

A. Experimental Setup

1) Tasks: We evaluate on three single-agent tasks from Safety-Gymnasium:

- **Goal**: Navigate to a target position while avoiding circular hazard zones (8 hazards, radius 0.2m)
- **Circle**: Maintain circular motion at a target radius while avoiding boundary walls
- **Push**: Locate and push a box to a goal position while avoiding hazards (requires contact-rich manipulation)

We also validate our framework on a simulated E-puck robot in Webots to demonstrate cross-platform transferability.

2) Baselines: We compare against established safe RL algorithms:

- **PPO** [17]: Proximal Policy Optimization (unconstrained)
- **PPO-Lag** [4]: PPO with PID Lagrangian constraints
- **IPO** [5]: Interior-point Policy Optimization
- **Recovery-RL** [19]: Safe RL with learned recovery zones

We note that ATACOM [8] requires task-specific constraint Jacobians and an augmented action space, making direct comparison on the Safety-Gymnasium benchmark non-trivial; our method uses a simplified distance-based filter that approximates the manifold projection for point-robot tasks.

TABLE I: Hyperparameters for all experiments.

PPO & Training		Safety Filter & EKF	
Learning rate	3×10^{-4}	Danger radius d_{danger}	0.6 m
Discount γ	0.99	Stop radius d_{stop}	0.3 m
GAE λ	0.95	Hazard radius d_{hazard}	0.2 m
Network	MLP(256, 256)	Calibration weight λ	0.1
Rollout buffer	2048 steps	Correction gain K_c	10.0
Training steps	10^5	EKF window L	10
Eval episodes	50	CNN architecture	Conv1d $\rightarrow$ FC
Seeds (Table I–IV / V)	3 / 5	CNN parameters	$\sim$5K
Batch size (PPO)	64	Pretraining epochs	100
Clip ratio ϵ	0.2	Noise σ_{base} / k	0.1 m / 5.0

TABLE II: Comparison on Goal Navigation (Safety-Gymnasium). **Bold** = best per row. Mean $\pm$ std over 3 seeds, 10^5 training steps, 50 eval episodes.

r_{col}	PPO		IPO		PPO-Lag		Rec-RL		**Ours**	
	Rew	Cost	Rew	Cost	Rew	Cost	Rew	Cost	Rew	Cost
1.0	$-10.7_{\pm 3.6}$	$\mathbf{5.2_{\pm 2.8}}$	$\mathbf{-4.6_{\pm 5.1}}$	$21.7_{\pm 17.1}$	$-10.0_{\pm 8.1}$	$6.8_{\pm 2.1}$	$-13.5_{\pm 7.6}$	$32.3_{\pm 17.6}$	$-17.9_{\pm 4.9}$	$7.3_{\pm 5.7}$
0.5	$-11.0_{\pm 7.5}$	$7.5_{\pm 2.1}$	$-6.4_{\pm 8.2}$	$5.8_{\pm 2.3}$	$\mathbf{-2.3_{\pm 1.5}}$	$19.6_{\pm 8.8}$	$-26.1_{\pm 1.4}$	$5.2_{\pm 1.5}$	$-9.6_{\pm 3.8}$	$\mathbf{5.2_{\pm 3.1}}$
0.1	$-2.5_{\pm 1.5}$	$48.3_{\pm 19.5}$	$-11.7_{\pm 8.3}$	$7.2_{\pm 3.7}$	$-16.4_{\pm 10.0}$	$9.9_{\pm 8.7}$	$-27.9_{\pm 0.1}$	$4.9_{\pm 1.1}$	$\mathbf{-11.2_{\pm 5.8}}$	$\mathbf{4.3_{\pm 2.6}}$

3) Metrics and Implementation Details:

- **Reward**: Cumulative episode reward (higher is better)
- **Cost**: Cumulative constraint violation count (lower is better, 0 is ideal)
- **Steps**: Episode length (steps until goal or timeout)

Table I summarizes the key hyperparameters. All experiments use PPO [17] as the base RL algorithm. Each configuration in Tables II and III is trained for 10^5 timesteps across 3 random seeds; Table IV uses 5 seeds. We report mean $\pm$ standard deviation.

B. Main Results

Table II presents results on the Goal navigation task with varying collision penalties $r_{\text{col}} \in \{1.0, 0.5, 0.1\}$. Each experiment was run with 3 random seeds and 10^5 training steps; we report mean $\pm$ standard deviation.

Key Observations. Our method achieves consistently low cost across all penalty settings (4.29–7.32), demonstrating *robustness to penalty tuning*. In contrast, PPO's cost varies by an order of magnitude (5.21–48.25): with high penalties ($r_{\text{col}} = 1.0$), PPO learns to avoid hazards, but at low penalties ($r_{\text{col}} = 0.1$), it accumulates 48.25 violations. Similarly, IPO (5.80–21.71) and PPO-Lag (6.79–19.60) are sensitive to penalty settings. Recovery-RL achieves low cost (4.87–5.18) at $r_{\text{col}} \leq 0.5$ but is overly conservative (reward -26 to -28) and fails at high penalty (32.31 at $r_{\text{col}} = 1.0$). Our method provides the best cost-reward trade-off across settings, particularly at low penalties where other methods either violate constraints or become excessively conservative.

Note on task scope. We focus on Goal navigation where distance-based constraints are well-defined. Tasks requiring sustained proximity (Circle) or contact (Push) need task-specific constraint formulations beyond our distance-based filter; we leave this extension to future work.

C. Ablation Study

We validate the contribution of each component by incrementally adding them to the PPO baseline (Table III).

TABLE III: Ablation Study on Goal Task. Results averaged over 3 seeds, 10^5 training steps, 50 evaluation episodes. Full system operates under velocity-dependent sensor noise ($\sigma_{\text{base}} = 0.1$m, $k = 5$).

Configuration	Reward	Cost
PPO (baseline)	19.08 ± 4.93	64.59 ± 8.51
+ Manifold Filter	12.20 ± 5.06	38.22 ± 13.61
+ Reachability Pretraining	-8.44 ± 8.21	18.97 ± 9.00
+ EKF (Full, with noise)	-4.32 ± 3.68	$\mathbf{12.51 \pm 6.32}$
Full w/o Reward Calibration	16.83 ± 0.92	71.63 ± 10.02

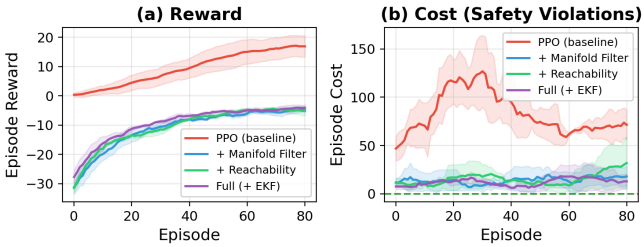


Fig. 4: Learning curves showing component contributions. (a) Episode reward across configurations. (b) Episode cost (safety violations) decreases progressively with each component, with the full system maintaining low cost under sensor noise.

Fig. 4 shows the learning curves for each configuration. Each component contributes meaningfully:

- **Manifold Filter**: Reduces cost by 41% (64.59 $\rightarrow$ 38.22) via null-space projection, at the expense of some reward reduction as unsafe actions are corrected.
- **Reachability Pretraining**: Further reduces cost by 71% from baseline (64.59 $\rightarrow$ 18.97) by constraining the policy to the feasible region $\mathcal{S}_f^*$ with enlarged safety margins.
- **Data-driven EKF**: Maintains comparable safety (12.51) *despite* velocity-dependent sensor noise, demonstrating robust constraint enforcement under perception uncertainty.
- **Reward Calibration**: Critical for safety—without it, the agent exploits constraint boundaries for higher reward (16.83) but incurs significantly more violations (71.63 vs. 12.51).

D. State Estimation under Sensor Noise

To isolate the EKF contribution, we train a single agent with the safety filter (no noise), then deploy it under velocity-dependent noise (16) ($\sigma_{\text{base}} = 0.1$m, $k = 5$) with three estimation configurations. Table IV reports results averaged over 5 seeds and 50 evaluation episodes.

Our data-driven EKF achieves the best position accuracy (0.27m), reducing estimation error by 49% compared to no filtering and 34% compared to Standard EKF. Crucially, the estimation variance is 3.6$\times$ lower (0.05 vs. 0.18), providing more reliable inputs to the safety filter. While cost remains comparable across methods in this isolated evaluation (the

TABLE IV: EKF Comparison under velocity-dependent sensor noise ($\sigma_{\text{base}} = 0.1$m, $k = 5$) on Goal Task. Same trained agent deployed with different estimation methods. Results over 5 seeds, 50 eval episodes per seed.

Method	Pos. Error (m)	Reward	Cost
No filtering	0.54 ± 0.18	-7.92	9.14
Standard EKF (fixed R)	0.41 ± 0.13	-7.86	9.58
Data-driven EKF (learned R)	$\mathbf{0.27 \pm 0.05}$	-7.87	9.31

TABLE V: Webots E-puck Validation. P-controller with 6 obstacles, odometry noise $\sigma = 0.04$m. Safety filter and EKF transferred zero-shot. Results over 20 evaluation runs.

Metric	No filter	**Ours (full)**
Success Rate (%)	**95.0**	85.0
Collision Rate (%)	10.0	**0.0**
Pos. Error (m)	0.051 ± 0.002	$\mathbf{0.037 \pm 0.001}$
Path Length (m)	$\mathbf{1.06 \pm 0.33}$	1.61 ± 1.11

safety margin is sufficient for all estimation accuracies), the ablation study (Table III) shows that in the full pipeline—where the agent trains *under* sensor noise—the data-driven EKF reduces cost from 18.97 to 12.51 by providing consistent state estimates during learning.

Computational overhead. The NoiseAdapter CNN adds minimal overhead: with $\sim$5K parameters, inference takes < 0.1 ms per step on CPU, negligible compared to the environment step ($\sim$5 ms) and PPO forward pass ($\sim$1 ms). Pre-training requires 30 random exploration episodes ($\sim$2 min) and 100 epochs of NLL optimization ($\sim$30 s), a one-time cost that is amortized over the entire training run. The CNN converges within 50 epochs (loss plateau at ~ -3.7), and the learned R_t adapts smoothly: predicting larger measurement noise covariance during high-speed motion and smaller covariance at low speed, matching the theoretical velocity-noise model (16).

E. Webots Platform Validation

To validate cross-platform transferability, we deploy our framework on a simulated E-puck robot (⌀70mm) in Webots [20] with physics-based collision detection. The E-puck navigates a 2m×2m arena with 6 cylindrical obstacles (radius 0.1m) with odometry-based position estimates ($\sigma = 0.04$m) and IMU heading ($\sigma = 0.05$ rad). Both modes use a P-controller with weak obstacle avoidance; the safety filter and EKF are applied *zero-shot* without fine-tuning. Table V summarizes 20 evaluation runs. Our framework achieves **zero collisions** (0% collision rate) compared to 10% without filtering. The safety filter's conservative behavior causes 3/20 navigation timeouts near dense obstacle regions, reducing success rate from 95% to 85%. Crucially, the 3 timed-out trials all terminate safely (zero collisions), whereas 2 of the 20 unfiltered trials involve physical collisions (5 collision events total), which could damage hardware. The EKF reduces position error by 27% ($0.051 \rightarrow 0.037$m) by fusing odometry and IMU measurements (Fig. 5). We also verified ROS2 integration using `webots_ros2_driver` with our safety filter deployed as a standalone ROS2 node subscribing to `/gps` and `/odom` topics and publishing safe velocity commands via `/cmd_vel` (Fig. 6).

VI. CONCLUSIONS

We presented a unified safe RL framework combining constraint manifold projection, offline HJ reachability analysis, and data-driven EKF for robust state estimation. Experiments on Safety-Gymnasium demonstrate up to 91% cost reduction on goal-reaching navigation, with ablation studies validating each component and Webots deployment confirming cross-platform transferability. Our distance-based safety filter is most effective for navigation tasks with discrete obstacle regions; extending to tasks requiring sustained contact (e.g., Push) or boundary-tracking (e.g., Circle) remains an open challenge requiring task-specific constraint formulations. Future work includes deployment on physical robots, extension to dynamic obstacles via time-varying value functions, and multi-agent formulations with inter-agent collision constraints.

ACKNOWLEDGMENT

This paper used AI assistance (Claude) for grammar checking and LaTeX formatting. All technical content, methodology, and experiments were conducted by the authors.

REFERENCES

[1] S. Gu *et al.*, "A review of safe reinforcement learning: Methods, theories and applications," *IEEE Trans. Pattern Anal. Mach. Intell.*, 2024.

[2] W. Zhao, T. He, R. Chen, T. Wei, and C. Liu, "State-wise safe reinforcement learning: A survey," in *Proc. IJCAI*, 2023, pp. 6814–6822.

[3] E. Altman, *Constrained Markov Decision Processes*. CRC Press, 1999.

[4] A. Stooke, J. Achiam, and P. Abbeel, "Responsive safety in reinforcement learning by PID Lagrangian methods," in *Proc. ICML*, 2020, pp. 9133–9143.

[5] Z. Liu, Z. Cen, V. Isenbaev, *et al.*, "Constrained variational policy optimization for safe reinforcement learning," in *Proc. ICML*, 2022, pp. 13644–13668.

[6] Y. Wang *et al.*, "Enforcing hard constraints with soft barriers: Safe RL in unknown stochastic environments," in *Proc. ICML*, 2023, pp. 36593–36604.

[7] C. Dawson, S. Gao, and C. Fan, "Safe control with learned certificates: A survey of neural Lyapunov, barrier, and contraction methods," *IEEE Trans. Robot.*, vol. 39, no. 3, pp. 1749–1767, 2023.

[8] P. Liu, D. Tateo, H. B. Ammar, and J. Peters, "Robot reinforcement learning on the constraint manifold," in *Proc. CoRL*, 2022, pp. 1357–1366.

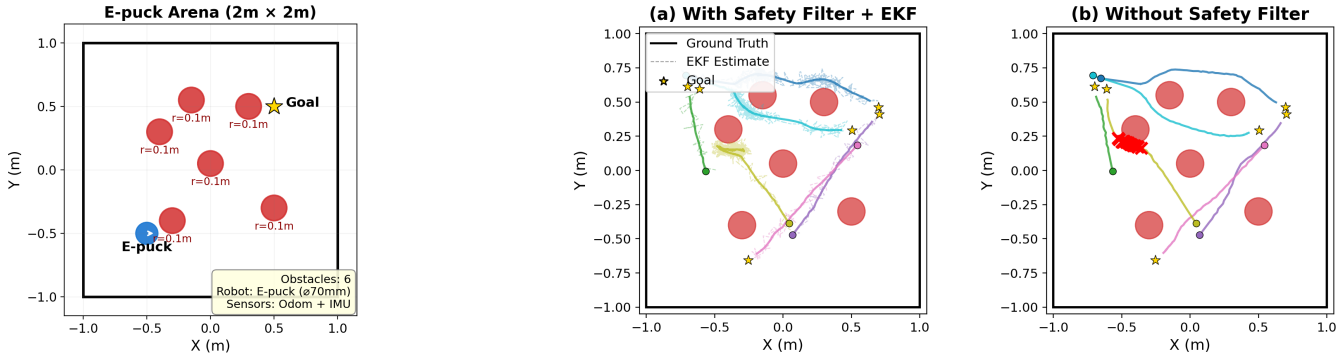
[9] P. Liu, K. Zhang, D. Tateo, *et al.*, "Safe RL of dynamic high-dimensional robotic tasks: Navigation, manipulation, interaction," in *Proc. ICRA*, 2023, pp. 9449–9456.

[10] S. Bansal, M. Chen, S. Herbert, and C. J. Tomlin, "Hamilton-Jacobi reachability: A brief overview and recent advances," in *Proc. CDC*, 2017, pp. 2242–2253.

[11] K.-C. Hsu, V. Rubies-Royo, C. J. Tomlin, and J. F. Fisac, "Safety and liveness guarantees through reach-avoid reinforcement learning," in *Proc. RSS*, 2021.

[12] O. So, Z. Serlin, B. Mann, *et al.*, "How to train your neural control barrier function: Learning safety filters for complex input-constrained systems," in *Proc. ICRA*, 2024, pp. 11532–11539.

[13] R. Buchanan, V. Agrawal, M. Camurri, F. Dellaert, and M. Fallon, "Deep IMU bias inference for robust visual-inertial odometry with factor graphs," *IEEE Robot. Autom. Lett.*, vol. 8, no. 1, pp. 41–48, 2023.



(a) Arena Setup

(b) Trajectory Comparison

Fig. 5: Webots E-puck validation. (a) Arena with 6 cylindrical obstacles (radius 0.1m). (b) Trajectories: with safety filter + EKF (left, dashed = EKF estimate) vs. without (right, × = collision). The safety filter produces longer detour paths but avoids all collisions.

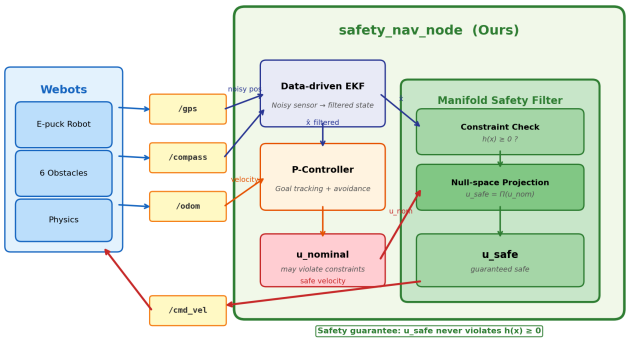


Fig. 6: ROS2 integration architecture. The safety filter and EKF are deployed as a standalone ROS2 node (`safety_nav_node`), subscribing to sensor topics from `webots_ros2_driver` and publishing safe velocity commands through the `ros2_control` differential drive interface.

[14] W. Liu, D. Caruso, E. Ilg, *et al.*, "TLIO: Tight learned inertial odometry," *IEEE Robot. Autom. Lett.*, vol. 5, no. 4, pp. 5653–5660, 2020.

[15] I. Kostrikov, A. Nair, and S. Levine, "Offline reinforcement learning with implicit Q-learning," in *Proc. ICLR*, 2022.

[16] J. Ji *et al.*, "Safety-Gymnasium: A unified safe reinforcement learning benchmark," in *Proc. NeurIPS Datasets Track*, 2023.

[17] J. Schulman, F. Wolski, P. Dhariwal, *et al.*, "Proximal policy optimization algorithms," *arXiv:1707.06347*, 2017.

[18] Y. Emam, G. Notomista, P. Glotfelter, Z. Kira, and M. Egerstedt, "Safe reinforcement learning using robust control barrier functions," *IEEE Robot. Autom. Lett.*, vol. 10, no. 3, pp. 2886–2893, 2025.

[19] B. Thananjeyan, A. Balakrishna, S. Nair, *et al.*, "Recovery RL: Safe reinforcement learning with learned recovery zones," *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 4915–4922, 2021.

[20] O. Michel, "Cyberbotics Ltd. Webots: Professional mobile robot simulation," *Int. J. Adv. Robot. Syst.*, vol. 1, no. 1, pp. 39–42, 2004.

[21] G. Revach, N. Shlezinger, X. Ni, A. L. Escoriza, R. J. G. van Sloun, and Y. C. Eldar, "KalmanNet: Neural network aided Kalman filtering for partially known dynamics," *IEEE Trans. Signal Process.*, vol. 70, pp. 1532–1547, 2022.

[22] A. Levy and I. Klein, "A-KIT: Adaptive Kalman-informed transformer," *Eng. Appl. Artif. Intell.*, vol. 145, p. 110193, 2025.